RAY TRACING AND BOUND BLOCK ELIMINATION METHODS FOR VIEWSHED ANALYSIS WITH EFFICIENCY ENHANCEMENTS THROUGH RADIAL SEGMENTATION AND PARALLEL PROCESSING

Adrian Widlak[1] and Jerzy Orlof[1]

[1]Cracow University of Technology, Faculty of Computer Science and Telecommunications, e-mail: adrian.widlak@pk.edu.pl, jerzy.orlof@pk.edu.pl

KEYWORDS

visualization of geographic (GIS); high performance computing; visibility analysis; point cloud analysis; LiDAR; computational geometry; viewsheds, modeling of a system processing a large point cloud; simulating visibility with digital data

ABSTRACT

Visibility analysis plays a key role in spatial planning and environmental protection, enabling the assessment of the impact of activities and supporting decision-making processes. The most accurate results in visibility analysis are obtained using point clouds from LiDAR technology, which are currently the most precise representations of the land surface. However, processing such data is computationally demanding and presents significant technological challenges. This article compares two methods of determining visibility: the classic ray tracing approach and an algorithm operating on a point cloud. Special attention is given to techniques that speed up the computation, such as parallel processing and dividing the point cloud into radial sectors. The accuracy and computational efficiency of the two methods were analysed, as well as their potential applications in practice. The results show that using radial partitioning significantly increases computational efficiency. A speed-up of 179% was achieved for the method using 200 sectors and 405% for a 300-sector split compared to a 100-sector split. The research presented here provides important insights for optimising visibility analysis in high-detail environments. The results obtained have relevance for the development of advanced decision-support tools in urban planning, environmental protection and other fields requiring precise spatial analysis.

INTRODUCTION

Viewsheds (Forczek-Brataniec, 2018) are used in many fields, including urban planning. In urban planning, visibility diagrams allow the impact of new developments on the urban landscape to be assessed, helping designers to integrate new developments harmoniously into existing spaces. They can also be used to analyse the visibility of key elements of the natural landscape, such as protected areas or historic sites, allowing them to be more effectively protected and minimising the negative impact of human activity. This enables more informed decision-making while accounting for social and environmental considerations.

Visibility in perspective and the ability to rotate a point cloud are crucial in many fields, especially in spatial analysis and design. Three-dimensional visibility models allow for a more intuitive interpretation of the data, providing a more accurate understanding of the spatial relationships between objects. The perspective representation of the data allows the visualisation of hidden relationships, such as the range of visibility from different heights or the effect of obstacles on limiting the field of view. Rotating the point cloud additionally enables detailed analysis from different directions, which is particularly useful for infrastructure design, urban planning or environmental analysis.

The processing of large areas is important due to the different accuracy requirements and the nature of spatial analyses (Zhao et al., 2013). Three levels of accuracy can be distinguished: urban, landscape, and planning. Due to the increasing demand for planning-scale analyses, there is a growing emphasis on studies covering large areas, which requires efficient methods for processing and optimising spatial data. Ongoing research focuses on adapting the Bound Block Elimination (BBE) (Widłak et al., 2024) algorithm to process the largest possible areas.

There is a limited number of studies analysing visibility on spatial graphs, which can be used to generate panoramic or orthographic perspective projections. Moreover, existing studies do not focus on parallelizing this process or attempting to obtain visibility as a point cloud results for extensive areas in a shorter time.

RELATED WORK

The development of visibility diagrams is a key aspect of spatial analysis and is carried out using various methods that vary in accuracy and computational efficiency. Traditional approaches based on raster methods and line-of-sight analysis in numerical terrain models (Fisher, 1993; De Floriani and Magillo, 2003) allow rapid determination of visible areas, but their precision is limited by

Communications of the ECMS, Volume 39, Issue 1,
Proceedings, ©ECMS Marco Scarpa, Salvatore Cavalieri,
Salvatore Serrano, Fabrizio De Vita (Editors) 2025
ISBN: 978-3-937 436-86-9/978-3-937 436-85-2
(CD) ISSN 2522-2414

the model's resolution. Alternatively, methods using ray-tracing, used, e.g., in computer graphics (Whitted, 1980; Glassner, 1989), enable detailed simulation of ray propagation in space and are widely used in visibility analyses (Johnson et al., 2015a).

In recent years, the development of LiDAR technology has significantly increased the possibilities for accurate surface and object modelling, allowing the indirect use of point clouds to generate visibility diagrams. White et al. (2013); Chen et al. (2021). Chen et al. (2021) have shown that visibility analysis from point cloud-based models allows even fine terrain structures to be considered, eliminating the limitations of interpolation used in raster models. In addition, visibility diagrams can be generated from Digital Elevation Models (DEMs) Wilson and Gallant (2000), which provide a uniform data structure and allow the use of advanced interpolation algorithms (Lee, 1994; Franklin, 2002). These methods allow for the effective identification of areas visible from any observation point, although their precision depends on the resolution of the DEM grid and the computational techniques used.

This paper is a natural extension of the author's previous research. It combines a method for parallelizing the slice based on the radial division of a point cloud (Orlof and Ozimek, 2021) and an algorithm for determining viewshed from point cloud with BBE method (Widłak et al., 2024). The need to investigate the two methods on the point cloud also arose as a result of our previous research, where the need to analyze the viewshed created on the point cloud was demonstrated due to the higher accuracy of the results thus (Orlof et al., 2024; Orlof, 2017). Compared to previous work, which focused mainly on the accuracy of visibility generation, the current research broadens the scope by attempting to parallelize (Pllana and Xhafa, 2017) algorithms to process extensive data ranges.

METHODOLOGY

For the analysis, data publicly available on geoportal.gov.pl was used, derived from the LIDAR resource PL-EVRF2007-NH, which refers to a digital elevation model (DEM). The data covers the area marked M-34-64-D-d-1-4-2-4 located in Krakow and shows the part near the Wawel Castle (Poland) with a density of 12 points/m² (Fig. 1). The area was scanned in 2023 (Geoportal, 2024; Zhou, 2017).

Creating viewsheds based on the ray tracing (Johnson et al., 2015b) algorithm involves simulating the propagation of light rays in three-dimensional space, which makes it possible to determine precisely which areas are visible from a given observation point and which are not. The process begins by defining the source of the rays and the geometry of the scene, which can include both terrain and obstacles such as buildings or trees. To make it, a three-dimensional terrain model is created in the form of a TIN (Triangulated Irregular Network) grid. The ray paths coming out of the observation point are then checked and their

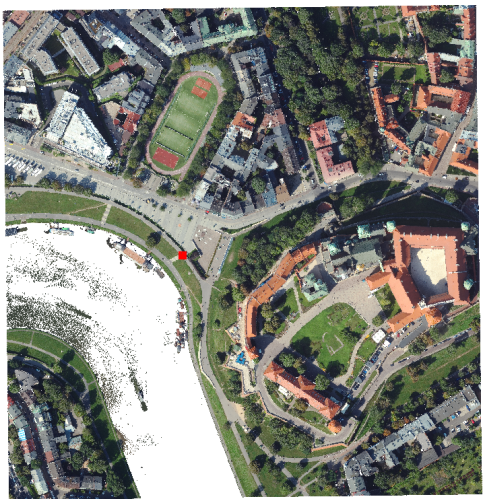


Figure 1: Input point cloud 6.351.545 points, 566,19m x 585,68m. Observer point is marked by red square.

interactions with objects in the scene analysed. Based on the ray tracing algorithm, it is possible to determine where the ray encounters an obstacle, thus determining whether the area is within the field of view.

The result of this process is a viewshed graph, which shows the distribution of visible and obscured areas in the analysed space in the form of a graphical orthogonal projection with visible and invisible spaces marked. Ray tracing offers high precision due to its ability to account for complex terrain. The main challenges with this method are the high computational power requirements, especially when analysing large scenes created on large-scale point clouds.

Creating viewsheds based on point cloud analysis using the BBE method (Widłak et al., 2024) involves separating visible and invisible points relative to a given observer point. This method provides spatial information about visibility, which makes it possible to generate graphs not only in an orthogonal top-down view but also in a real-time perspective view. Data prepared in this way allows visibility analysis at any point in space and, in addition, by positioning the view relative to the observer, it is possible to obtain a panoramic representation of visibility from the perspective of the observation point. This is a significant advantage over previously used methods.

The algorithm for selecting visible and invisible points in a point cloud works by analysing the surface topology based on the triplets of points belonging to the surface and relative to a given *observer point*. Its main objective is to determine which points of the cloud are in the area visible from the observer's perspective and which are obscured by other elements of the surface.

The first step of the algorithm is to create vectors directed from the observer point to each of the vertices of the triangle defining the surface:

$$V_i = P_i - \vec{P}_{\text{observer}}, \quad i = 1, 2, 3 \tag{1}$$

Based on the above vectors, the normal vectors N_1, N_2, N_3 are computed, representing the planes formed by the triangle vertices and the observer point. These vectors are obtained as *cross product* between pairs of vectors:

$$N_1 = V_1 \times V_2, \quad N_2 = V_1 \times V_3, \quad N_3 = V_2 \times V_3. \quad (2)$$

Next, for each plane, the values of the *scalar products* between the normal vectors N_i and the third vector V_i are determined.

$$s_1 = N_1 \cdot V_3, \quad s_2 = N_2 \cdot V_2, \quad s_3 = N_3 \cdot V_1. \quad (3)$$

For each analyzed point in the cloud, analogous scalar products are calculated with respect to the vector connecting the point to the observer. The signs of these products are compared with the signs of the reference values s_i, which allows determining whether the point lies on the same side of the plane as the triangle.

Additionally, the algorithm defines the *side vectors* (V_{side1}, V_{side2}) of the triangle:

$$\vec{V}_{\text{side 1}} = P_2 \vec{-} P_1, \quad (4)$$

$$\vec{V}_{\text{side 2}} = P_3 \vec{-} P_1, \quad (5)$$

and computes their normal ($\vec{N}_{\text{side}}$)) using the cross product:

$$\vec{N}_{\text{side}} = \vec{V}_{\text{side 1}} \times \vec{V}_{\text{side 2}}. \quad (6)$$

The vector side_normal represents the orientation of the triangle's plane. The scalar product between side_normal and the vector connecting the observer's point to the first vertex determines whether the analyzed point is visible. If the signs of the scalar products do not match, the point is considered invisible.

However, the disadvantage of the BBE method is the high computational cost. The generation of a spatial visibility map requires prior sorting of the points in the cloud in relation to the observer, the preparation of information about the topology of the points (by grouping them into faces), and then the application of algorithms to eliminate points that are not visible with respect to the observer. The algorithm requires each point in the cloud to be processed relative to a selection of the nearest three points defining the surface. This means that its computational complexity is proportional to the number of all points and three points belonging to the considered face. However, the high precision of the analysis and the ability to generate spatial visibility maps make the algorithm particularly useful for assessing visibility in natural environments. The above problems can be solved by using an appropriate parallelization or point cloud partitioning strategy.

The radial division of the point cloud is based on the segmentation of space in the form of radially spreading sectors, with a central observer point (Orlof, 2017). Each sector is a radial slice of a defined point cloud. This allows the method to maintain the spatial relationships between points relative to a selected observer point, which is crucial in visibility analyses using the raytracing method. This subdivision allows the geometry of the terrain and objects to be accurately represented in space, preserving the continuity of the terrain required for ray analysis.

The radial method starts by determining the central point from which the sectors diverge. The number of sectors is then determined. As a result, each sector contains points within a certain angular range, which makes it possible to create partial visibility diagrams for individual areas. Once the calculation is complete, the results from the individual sectors are merged to create a complete visibility diagram that faithfully represents the actual spatial conditions.

RESULTS

In this section, we present the visibility analysis conducted on point cloud using two different methods: Ray tracing and Bound Block Elimination (BBE). This analysis aims to evaluate and compare how each approach determines the visibility of points within a given scene.

For ray tracing, we provide an orthographic viewshed, which offers a top-down view of the visibility distribution across the point cloud (Fig. 2). This representation effectively illustrates the areas classified as visible (white color) and occluded (black pixels) based on the interaction of rays with the cloud's geometry.

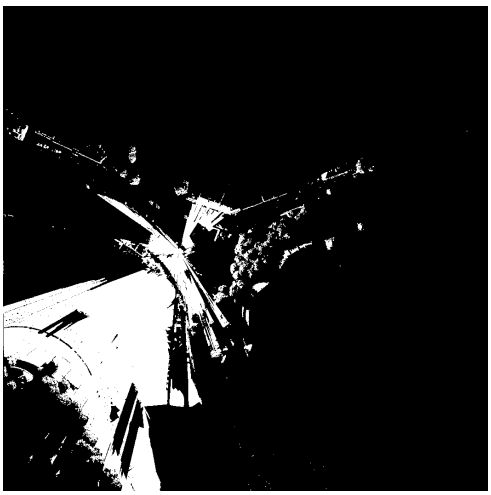
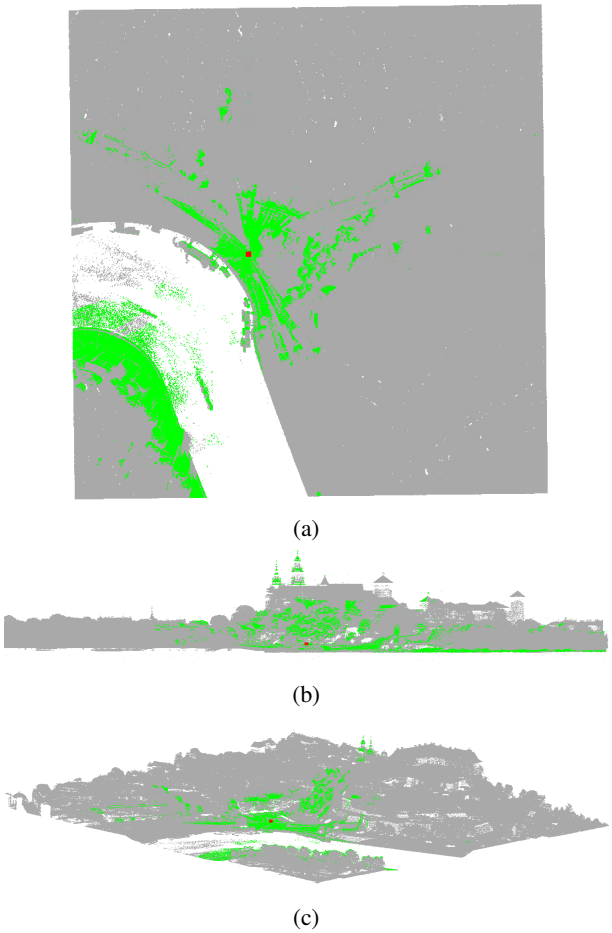


Figure 2: Ray tracing top ortho view. Created on the TIN surface generated from the point cloud

For BBE, we extend the visualization to include multiple perspectives. In addition to an orthographic viewshed, we also present a perspective view and a panoramic projection, providing a more comprehensive understanding of the visibility structure within the scene (Fig. 3). These additional visualizations highlight the efficiency and accuracy of the BBE method in resolving visibility constraints.



(a)

(b)

(c)

Figure 3: Bound Block Elimination algorithm executed on point cloud (Figure 1). Observer point - red, visible point - green, hidden points - gray **(a)** - top view; **(b)** - left view; **(c)** - perspective view

Table 1: Number of points visible compared to all points in each of the discussed methods (ray tracing, Bound Block Elimination)

Method	**Visible**	**Total**	**%**
RAY TRACING	130.071 pixels	1.113.025 pixels	11,69%
BBE	265.899 points	6.260.470 points	4,25%

The results obtained from both methods are analyzed to assess their effectiveness in visibility computation, considering factors such as computational efficiency and accuracy in handling occlusions. Table 1 shows that the visibility analysis conducted with ray tracing indicates 11,69% visibility, while the same area analyzed with the BBE method indicates 4.25% visibility. This comparison suggests that when considering points in 3D coordinates (x, y, z), the visibility is lower compared to assessing an orthogonal top-down view from a map generated using ray tracing. The first method provides a generalized result, whereas the second method offers information about individual points and additional third dimensions, requiring further analysis to determine the visibility relationship between them.

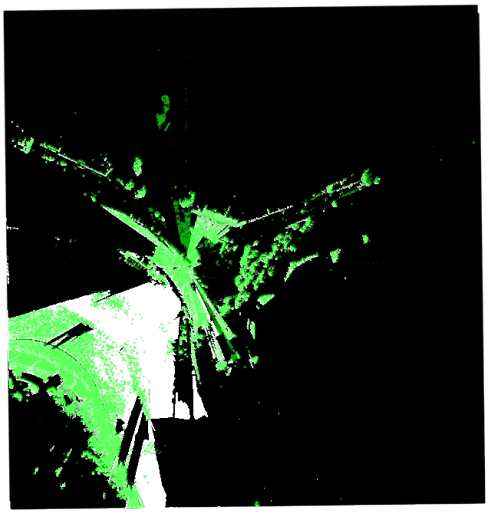


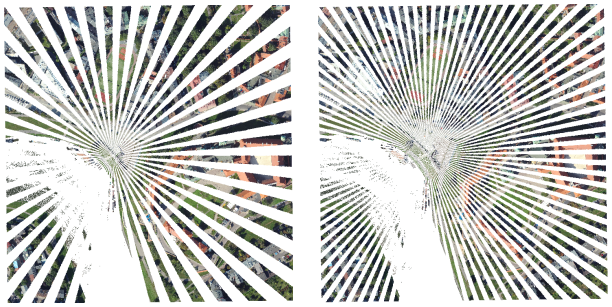
Figure 4: BBE point cloud based viewshed graph (green) imposed on ray tracing viewshed graph (white)

The overlaid viewshed graphs in the orthogonal view exhibit significant differences. This is primarily due to the fact that the green graph (Fig. 4) is rendered based on points, whereas the white one is based on TIN surface. The white area that is not represented in the green part corresponds to the river—since the LIDAR scanning did not capture a large number of points in this region due to a lack of reflection, visibility is not shown. However, by adding triangulation to the obtained visibility point cloud, we can observe full visibility in this area - the same as is received from the ray tracing method. Interestingly, in the upper region, BBE revealed a large fragment that was not detected with the ray tracing method, indicating the studied method's high precision.

The computation time for generating a viewshed using the Ray-Tracing method for the given point cloud was 256 seconds, while the BBE algorithm is not able to generate a visibility analysis for such an area. Nevertheless, the assumption is that we want to obtain a panoramic viewshed graph from the observer's point of view and to view the visibility in perspective, so our research is focused on trying to obtain a viewshed using radial segmentation of the cloud in order to obtain results as quickly as possible. In addition, the processing of the separated sections into available processes in the system was paralleled to speed up the calculations.

In accordance with the chosen method of dividing the point cloud from the observer's point into radial sections, we decided to carry out tests on the division of the input cloud into 100 and 200 radial sections, respectively (Fig. 5). To validate the results, a division into 300 sectors was also performed.

A viewshed generation method based on the BBE algorithm was performed on each slice, resulting in a radial



(a) (b)

Figure 5: Dividing the point cloud into radial sections. **(a)** - 100 sections, the graph shows even slices 0, 2, 4, ..., 98; **(b)** - 200 sections, the graph shows even slices 0, 2, 4, ..., 198

slice with filtered visible and obscured points for the selected observer point, slice processing time, data preparation time for the slice (Fig. 6, Fig. 8, Fig. 7, Fig. 9).

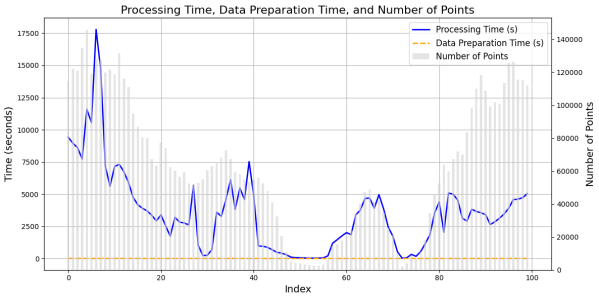


Figure 6: Processing time chart for calculation based on 100 radial division of point cloud. Max value for processing time is 17796,32 seconds and min value is 22,25 seconds

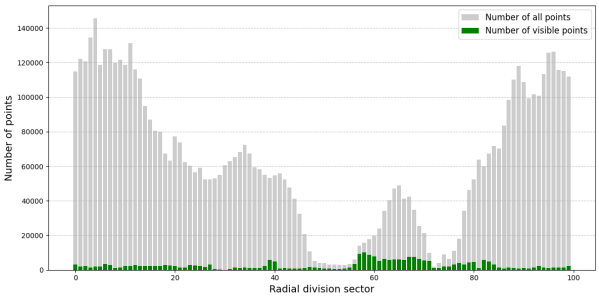


Figure 7: Distribution of all points and visible points across 100 radial sectors in point cloud

DISCUSSION

The graphs show that as the number of points in a sector increases, the processing time also increases (Fig. 6, Fig. 8). In some cases, processing time increases with the number of points within a sector (Fig. 7, Fig. 9) however, in others, this trend is not observed. This variation is

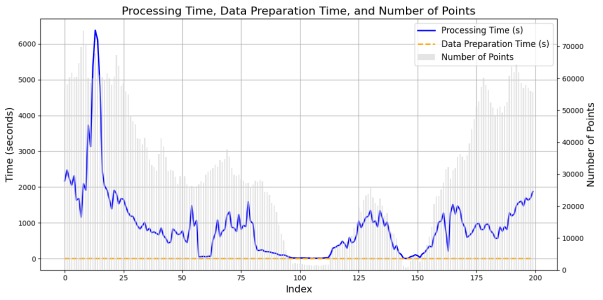


Figure 8: Processing time chart for calculation based on 200 radial division of point cloud. Max value for processing time is 6377,95 seconds and min value is 2,87 seconds

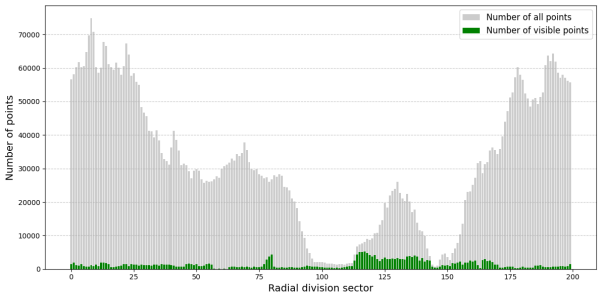


Figure 9: Distribution of all points and visible points across 200 radial sectors in point cloud

attributed to the non-uniform distribution of points. When a higher concentration of hidden points is located closer to the observer, the remaining points are more likely to be occluded as well. As a result, the algorithm can quickly eliminate them from further processing, thereby reducing computation time.

The calculations were performed on a computer with an Intel(R) Core(TM) i9-14900K processor 3.20 GHz, 64.0 GB RAM, a 64-bit operating system, and an x64-based processor.

To optimise the processing of a large number of files, the implementation was parallelized using the multiprocessing library in Python. The process retrieved each slice from the radial split, then, using a pool of processes, parallel processing of these slice was initiated. The number of processes was dynamically adjusted to the number of available CPU cores. Each slice of the cloud was processed separately, allowing for rapid load distribution and efficient use of available processing power. With this solution, file processing times were reduced compared to the traditional approach, in which files would be processed one after the other.

For a cloud divided into 100 sectors, the total execution time for visibility on the cloud was 352,76 minutes, or 5 hours and 52 minutes. For a cloud divided into 200, the execution time for visibility on the cloud was 126,12 minutes, or 2 hours and 6 minutes. This is the total algorithm processing time for all points across the cloud.

For the division of the point cloud into 100 radial segments, the processing times recorded were MAX =

17796,32 seconds and MIN = 22,25 seconds. In contrast, when the cloud was divided into 200 radial segments, the processing times significantly decreased to MAX = 6377,95 seconds and MIN = 2,87 seconds.

This demonstrates a notable improvement in computational efficiency as the granularity of segmentation increased. Specifically, the reduction in the maximum processing time between the two cases was approximately 11418,37 seconds, which corresponds to a difference of 190,31 minutes.

$$\Delta_{\text{MAX, minutes}} = \frac{17796,32}{60} \approx 296,61 \text{ minutes}$$

Such results highlight that the optimization of the algorithm not only enhances its performance for larger segmentations, but also exhibits remarkable improvements in scenarios with minimal processing times.

Ratio of processing times for the maximum value of two divisions:

$$A_{\text{MAX}} = \frac{t_{100,\text{MAX}}}{t_{200,\text{MAX}}} = \frac{17796,32}{6377,95} \approx 2,79$$

The algorithm has sped up by **179%**.

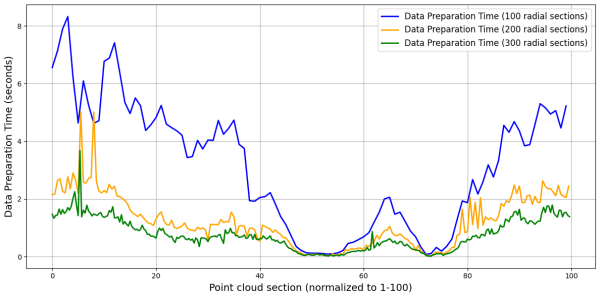


Figure 10: BBE visibility processing time for different point cloud radial segmentation

To further validate the adopted method, the input data was further subdivided into 300 radial parts. Times were measured, with a maximum processing time of 3522,21 seconds and a minimum of 1,25 seconds. The entire visibility process took 74,33 minutes. A summary of the times is shown in fig. 10.

$$A_{\text{MAX}} = \frac{t_{100,\text{MAX}}}{t_{300,\text{MAX}}} = \frac{17796,32}{3522,21} \approx 5,05$$

The algorithm has sped up by **405%**.

The main limitation of the algorithm is the amount of available RAM. Depending on the size of the processed point cloud, all points within a given section must be loaded into memory simultaneously. In the case of large-scale point clouds, this is often not feasible on commonly available computers, which typically offer an average of 32 GB of RAM.

SUMMARY AND FUTURE WORK

In this research, we explored two methods for determining visibility: viewshed analysis based on point clouds (BBE) and ray tracing. The results indicate that viewshed determination using a point cloud is highly promising, as it allows one to generate various visualization outputs, such as perspective views, panoramic images, and top-down orthogonal maps. These visual representations provide a more comprehensive way to analyze spatial relationships and better understand the observer's perspective. In comparison, ray tracing generates a single image of an orthophoto map, while the point cloud analysis with BBE creates a point cloud with marked visible and hidden points.

However, due to computational time constraints, faster processing methods are necessary. To address this, we analyzed an approach based on dividing the scene into radial slices, referred to as sectors, originating from the observer's viewpoint. The greater the number of sectors, the faster the processing time, as each slice can be computed independently, reducing the overall complexity. This sector-based division significantly accelerates visibility computations and enables the generation of 3D viewsheds. The method demonstrated a performance improvement of 179% with 200 sectors and 405% with 300 sectors compared to the 100-sector split.

Further research will analyze additional cases and spatial situations to better understand how segmentation influences the accuracy of viewshed analysis. We will also investigate the point at which dividing the scene into more sectors no longer provides faster processing times.

In future work, we plan to integrate the radial sector approach with the BBE algorithm to discard points based on their radial coordinates dynamically - in the current example, we tested two approaches synchronously, while we want to integrate them and execute them together. This optimization will improve the processing efficiency, making visibility analysis even faster and more scalable. In addition, we will refine our understanding of the relationship between sector count and processing time to ensure that the method is efficient and accurate under varying conditions.

This advancement can transform urban planning workflows, landscape protection, and spatial analysis. By leveraging point cloud data instead of single 2D graphs, this approach facilitates faster and more precise visibility assessments, ultimately improving geospatial (GIS) analysis decision-making.

References

Xiaozhi Chen, Jian Wang, David Held, and Martial Hebert. Panonet3d: Combining semantic and geometric understanding for lidar point cloud detection. *arXiv preprint arXiv:2012.09418*, 2021.

Leila De Floriani and Paola Magillo. Visibility and traversal of triangulated terrain models. *Computers & Graphics*, 27(5):717–731, 2003.

Kevin D Fisher. Placing social interaction: an integrative approach to analyzing past built environments. In

Proceedings of the 24th Annual Conference of the Environmental Design Research Association, pages 111–117, 1993.

Urszula Forczek-Brataniec. *Visible space. A visual analysis in the landscape planning and designing*. Wydawnictwa Politechniki Krakowskiej, 9 2018.

William R. Franklin. Raster-based visibility algorithms for topographic analysis. *Computers & Geosciences*, 28(8):965–976, 2002. doi: 10.1016/S0098-3004(02)00024-7.

Geoportal. Geoportal krajowy. `https://mapy.geoportal.gov.pl/imap/Imgp_2.html`, 2024. Accessed: 20 December 2024.

Andrew S. Glassner. *An Introduction to Ray Tracing*. Academic Press, San Diego, CA, 1989. ISBN 978-0122861604.

Chris Johnson et al. *Feature Visibility Analysis*, pages 65–85. Springer, 2015a.

Gregory S. Johnson, Ingo Wald, Sven Woop, Carsten Benthin, and Manfred Ernst. High-performance ray tracing. *High Performance Parallelism Pearls: Multicore and Many-core Programming Approaches*, pages 349–358, 2015b. doi: 10.1016/B978-0-12-802118-7.00021-2.

Joseph Lee. Visibility analysis in geographic information systems. *International Journal of Geographical Information Systems*, 8(1):1–15, 1994. doi: 10.1080/02693799408901921.

Jerzy Orlof. Point cloud based viewshed generation in autocad civil 3d, 2017.

Jerzy Orlof and Paweł Ozimek. Tin surface and radial viewshed determination algorithm parallelisation on multiple computing machines. *Symmetry*, 13(3), 2021. ISSN 2073-8994. doi: 10.3390/sym13030424. URL `https://www.mdpi.com/2073-8994/13/3/424`.

Jerzy Orlof, Paweł Ozimek, Piotr Łabedz, Adrian Widłak, and Agnieszka Ozimek. Generating viewsheds based on the digital surface model (dsm) and point cloud. *PLOS ONE*, 2024. Currently under review.

S Pllana and Fatos Xhafa. *Programming Multicore and Many-Core Computing Systems*. John Wiley & Sons, 10 2017. ISBN 9780470936900. doi: 10.1002/9781119332015.

Joanne C White, Michael A Wulder, Andres Varhola, Mikko Vastaranta, Nicholas C Coops, Bruce D Cook, Doug Pitt, and Matt Woods. Lidar-derived measures of hurricane-affected forest area and damage severity in a subtropical wet forest in puerto rico. *Remote Sensing of Environment*, 134:210–223, 2013.

Turner Whitted. An improved illumination model for shaded display. *Communications of the ACM*, 23 (6):343–349, 1980.

Adrian Widłak, Paweł Ozimek, Piotr Łabedz, and Jerzy Orlof. Bound block elimination algorithm for accurate visibility computation in point cloud analysis. In *Proceedings of the 38th European Council for Modelling and Simulation Conference (ECMS 2024)*, 06 2024. doi: 10.7148/2024-0490.

D.J. Wilson and J.C. Gallant. Digital terrain analysis. In D.J. Wilson and J.C. Gallant, editors, *Terrain Analysis: Principles and Applications*, pages 1–27. John Wiley and Sons, New York, 2000.

Yanli Zhao, Anand Padmanabhan, and Shaowen Wang. A parallel computing approach to viewshed analysis of large terrain data using graphics processing units. *International Journal of Geographical Information Science*, 27:363 – 384, 2013. URL `https://api.semanticscholar.org/CorpusID:28907974`.

Qiming Zhou. Digital elevation model and digital surface model. *International Encyclopedia of Geography: People, the Earth, Environment and Technology*, pages 1–17, 2017.

AUTHOR BIOGRAPHIES



ADRIAN WIDLAK graduated with a degree in Computer Science from Cracow University of Technology in Poland, earning a B.Eng. and an M.Sc. with a specialization in Computer Graphics and Multimedia. Since then, he has been actively engaged as a researcher and lecturer at the Faculty of Computer Science and Telecommunications at the Cracow University of Technology. He participated in the TopMinds mentoring program, supported by the Top 500 Innovators Association and the Polish-American Fulbright Commission. His interests include point cloud processing, artificial intelligence, computational geometry, video image processing, and data visualization.



JERZY ORLOF completed his computer science studies with a computer graphics specialization at the Cracow University of Technology, earning the title of MSc in 2015. He expanded his collaboration with the university by becoming a teaching assistant at the Institute of Computer Science. His research focuses on CAx software, computer graphics, and point clouds. He specializes in creating viewsheds based on large-scale point clouds. Jerzy Orlof has acquired extensive knowledge and professional experience as a long-time certified instructor for Autodesk. He is the author of many publications related to point cloud processing.